The State Machine Is the Policy: A Scripted Chatbot for Federal Consultations over WhatsApp

Abstract

In Brazil, WhatsApp is civic infrastructure: a public consultation that exists only on the web cannot reach the whole public. This article presents a case study of decidim-chatbot, the engine that extends the Brazilian federal participation platform, a multi-tenant Decidim deployment, into WhatsApp, operated through SERPRO, the federal government's sanctioned messaging provider. The bot is scripted rather than conversational: a YAML state machine defines the entire dialogue, product staff edit message variables under length constraints, and structural changes go through code review and deployment. We describe the engineering that keeps a scripted bot reliable at scale: per-phone serialization and rate limits, idempotent delivery with staged encrypted replies, one-time bearer links that separate identity from conversation, and vote casting that reuses Decidim's own permissions and commands, so that a vote cast in chat goes through the same command as a vote cast on the site. We discuss the trade-offs of deterministic dialogue in government and the governance properties of a conversation whose policy is its source code. The engine is open source under AGPL-3.0-or-later.

1 Introduction

Brasil Participativo [1], the Brazilian federal government's citizen-participation platform, is a multi-tenant SaaS deployment of Decidim 0.32.x [2, 3] operated by the Presidency of the Republic and developed by the Lab Livre team at the University of Brasília. It has roughly two million registered users. Like most participation platforms, it is web-native, but in Brazil the channel the public actually uses, across income, age and geography, is WhatsApp. A consultation available only as a website excludes the audience that participation policy most needs to reach.

decidim-chatbot is the engine that addresses this gap. It is hosted in the platform's main repository [1] as a self-contained Decidim engine. It serves participants over WhatsApp through SERPRO's sanctioned government messaging API [4], lets them choose among the proposals of a published consultation, casts their vote through Decidim's own voting machinery, and reports a live ranking computed across site and chat. This article is a case study of that engine. It concentrates on the design decisions that make a chatbot defensible as an instrument of public consultation; these decisions are recorded in the source code itself, requirement by requirement. Section 2 states the need, Section 3 describes the architecture, Section 4 walks through a use case, and Sections 5 to 8 cover limitations, related work, availability and conclusions.

2 Statement of need

The need to meet citizens where they are is easy to state, but four constraints make it demanding.

The channel is informal, but the act is formal. Voting in a federal consultation must satisfy one-vote-per-person, respect the consultation's step-level settings, and produce an auditable record. A chatbot that approximates voting with free text would be quick to build, but it could not be defended in operation.

The provider is fixed by the government. A federal system cannot route citizen messages through a commercial conversational-AI vendor on foreign infrastructure. The platform must run on the government's sanctioned messaging channel, which in Brazil is SERPRO's WhatsApp API [4]. That channel constrains message formats (buttons, lists, call-to-action links) and imposes size limits that the dialogue must respect.

The methodology is flexible, and the knowledge is shared. Designing a participatory process requires methodological expertise that is not always available to the organizations implementing it. The chatbot embeds a ready-to-use participatory workflow grounded in accumulated empirical experience, which turns methodological knowledge into a shared, reusable resource. Organizers start from a preconfigured process that can be adapted to different contexts while preserving minimum methodological standards, instead of designing a consultation from scratch. This lowers barriers to adoption, supports consistent participation practices, and makes collectively developed knowledge available through the software itself.

The conversation is content, and content is governance. Every sentence the bot sends is an official communication. Someone must own it, review it and version it, and must be able to answer, years later, what exactly the government said to a given citizen. Neither a machine-learning model's weights nor a CMS edit history answers that question well.

3 Architecture and features

3.1 Overview: from webhook to reply

An inbound WhatsApp message arrives at a webhook endpoint, is normalized by a pluggable broker layer into the engine's internal event format, and is stored as a pending inbound event. A background job (`ProcessInboxJob`, running on Sidekiq) works through each phone's pending events, oldest first. For each event, a state machine advances that phone's conversation through the scripted flow, any vote is cast through Decidim's core commands, and the reply messages are staged as outbound events before being delivered through the broker. The production broker, `SerproWhatsapp`, implements sending, delivery status polling and webhook registration against SERPRO's API, with client credentials held as per-organization settings.

The broker abstraction extends beyond SERPRO. Every channel goes through the same `HandleMessage` path, so the conversation logic contains no provider code, and the terminal simulator used in development exercises the same machinery as a phone.

3.2 The conversation as a governed artifact

The dialogue is a YAML file, a fixed script transcribed from the product team's consultation document. It declares states, per-state messages in WhatsApp's four formats (text, buttons, list, call-to-action URL), transitions, fallbacks and escalation behavior. Three properties make it governable.

Separation of text and structure. Administrators edit only variables: named short terms (platform name, responsible entity, audience, location) and whole messages, each under an explicit `max_length`. Everything else, including states, transitions and branching, changes through code review, deployment and a `chatbot:validate` rake task that checks structure and WhatsApp limits. The review process for what the government says is the pull request.

Deterministic escalation. The script defines exactly what happens when input is not understood. Table 1 summarizes the policy, which sets the limit on how much misunderstanding the bot tolerates. The bot never loops on apologies, and every input has a defined outcome.

Table 1. Misunderstanding policy, as scripted.

Trigger	Bot behavior
First unrecognized input in a state	Re-send the state's messages (fallback)
Second consecutive miss	Escalation notice, then the fallback
Audio, image or other media	Dedicated notice: buttons only
Third consecutive miss onward	Silence

Defined lifecycle edges. States declare `if_empty` (where a list goes when there is nothing to vote on), `requires_vote` (where a confirmation goes when the chosen proposal is no longer available, which drops the stale vote), and `final` (the conversation is over and the script does not restart). Returning participants land in a dedicated state that remembers their participation and offers sharing or closing. YAML anchors share repeated blocks, so the whole file reads as the product document it implements.

3.3 Ordering, idempotency and abuse control

A messaging channel raises problems that a web form does not, and the job layer addresses each of them concretely.

- **Interleaving.** Two quick messages from one phone must not interleave. A per-phone lock serializes jobs, and events are processed in arrival order.
- **Abuse.** A phone may receive at most 30 bot replies per hour. Past that limit, its messages still advance the conversation but receive no answer, which caps both nuisance and runaway loops (a full conversation takes about a dozen replies).
- **Retry safety.** Each inbound event and its replies commit together, with replies staged encrypted in the outbox and delivered only after commit. A transient provider failure retries delivery of the staged rows without applying the same input to the conversation twice.
- **Observability with privacy.** When anything fails, only the error class is stored, never the message body or the phone number, because provider error responses can echo either and the logs of a participation system should not become a store of citizen communications. Delivery status is polled from the provider by request id, so a sent-but-undelivered reply can be distinguished from one the provider never accepted.
- **Fault isolation.** An event that raises is marked failed and does not block the phone's later messages. A deactivated broker drains its pending events as ignored rather than half-answering them.

Each of these is a small decision. Together they determine whether the bot can be operated by a ministry rather than only demonstrated.

3.4 Identity without registration

The hardest problem is establishing that a vote cast from a phone belongs to a person entitled to cast it, without sending that person through a web registration they are unlikely to complete. The engine's answer is a one-time identification handoff.

When the script reaches the step that validates the vote, the conversation receives a call-to-action link issued as an identity link. The link carries a bearer secret of 32 random bytes, of which only a SHA-256 digest is stored. It is valid for one hour, burnt on the first successful submit, locked after five wrong attempts, and invalidated wholesale when a new link is issued. The link opens a lightweight identification page with two modes, chosen per consultation: the participant either informs name and CPF (Brazilian tax ID) as an ephemeral participant, an account that exists only for this act and follows Decidim's ephemeral-participation line of work [5], or authenticates with the national GovBR single sign-on. The identification service that orchestrates this never sees the CPF itself; the credentials stay inside the GovBR handler.

The outcome taxonomy is explicit. A participant created for this vote who fails to vote is rolled back entirely, while one who already existed (because they voted on the site, or from another phone) is kept. A CPF that belongs to a registered account cannot be silently adopted by an ephemeral session; those citizens are told, in the conversation itself, that this identity votes on the site. The conversation records the vote only when it actually went through, which is also what the administrative dashboard counts.

The design is also a data-minimization strategy in the sense of Brazil's LGPD. The phone number, a durable personal identifier, stays inside the messaging layer and is referenced only as an opaque per-broker user identifier that the conversation layer never enriches. The identification page collects CPF and name only to perform the authorization, through a handler that the conversation services never see. The link that ties the two together exists for one hour, works once, and afterwards is a burnt row of hashed bytes. No single table joins a phone number to a tax ID, and the architecture is built to keep it that way.

3.5 One vote, two channels

Casting the vote is delegated entirely to Decidim: the chat uses the same permission class and the same VoteProposal command as the web UI, executed under a database lock on the participant that serializes concurrent attempts across devices. The chat adds a transport but no parallel voting semantics. The outcome taxonomy (voted, already_voted, closed, not_allowed, unavailable) maps one-to-one onto conversation events, so the citizen is told in plain language that their vote was counted, that they had already voted, or that voting has closed, and never receives a raw error. The live ranking rendered after confirmation is computed after the vote is saved, so it already counts the vote just cast. Site and chat votes aggregate into a single ranking because they are a single dataset.

The proposal list is assembled with the channel's constraints in mind. It draws from the published, non-withdrawn, non-rejected proposals of the consultation component the administrator picked, capped at the number of rows a WhatsApp list message holds. Titles and descriptions are stripped of HTML (WhatsApp renders no markup, and double-escaped entities would show literally) and truncated to the provider's field limits, in the participant's own locale. The administrative dashboard tells the operator how many proposals fall outside the list, so the channel's ceiling is a visible product decision and not a silent data loss.

3.6 Operating surface and multi-tenancy

Because the platform is a SaaS, every organization runs its own script instance, broker configuration and message variables, and inbound processing resolves the organization and its settings per message. The administrative surface follows the platform's conventions: a setup wizard, broker connection tests that talk to the provider's API directly, a script publication flow with preview paths that exercise the state machine end to end (including off-script inputs and refusal branches), and the terminal simulator already mentioned. The script's sample options exist so that previews and validation work before any real consultation is connected.

4 Use case: a consultation over WhatsApp

The script's happy path, as a citizen lives it, runs as follows. A greeting names the platform and the responsible entity and asks whether the person wants to participate. Choosing yes presents the consultation's proposals as a WhatsApp list, and a confirmation step asks the citizen to validate the choice. The validation link arrives as a call-to-action button and opens the identification page. Once the citizen is identified, the vote is cast through Decidim, and the citizen receives a confirmation plus the live top-three ranking. A final state offers a shareable message so that participation propagates through the same channel it arrived by. Declining, asking for more information, changing one's vote, an expired link, an already-voted identity and a closed vote each have a scripted state. The preview paths in the repository walk a reviewer through all of them, including the off-script path, where a participant types greetings and questions at the bot for ten messages before touching a button, and the login-help path, where a lost link is reissued, cancelled and validated in sequence. Previews are not mock-ups: the administrator's text edits are substituted into the same state machine that participants run, against sample options, so the text approved in review is the text that ships to phones.

Running such a consultation is an operational act as well as an editorial one. The broker connection is tested against the provider's API before launch, the webhook is registered with correct basic-auth credentials, the script is published from its draft, and the dashboard then tracks what arrived, what was answered, and which phones hit the hourly reply ceiling.

The full journey was field-validated on September 17, 2026, with the university community of the University of Brasília. The team stood up a short-lived test environment hosting a simulated consultation, with proposals for how the academic community imagines a better university, and distributed it through the mechanism the script's share step envisions: a single wa.me link, posted to student groups, that opens WhatsApp with the consultation's greeting ready to send. Instructors coordinated a synchronized session with their classes, so the validation served as a load test as well as a usability test. The exercise also exposed the channel constraint that the script's max_length discipline addresses: the first version of the launch message exceeded WhatsApp's limit for the collapsed "read more" view and had to be shortened before distribution. The limits enforced at validation time on the bot's own texts applied equally to the texts written by humans.

This journey ran within the Brasil Participativo program, alongside the platform's paragraph-level consultation components. Long-form normative documents are the content most often pushed to WhatsApp audiences, and the paragraph granularity at which the platform stores such documents is what makes chat-based reference to specific articles workable. We do not report engagement metrics, because production analytics are outside

the scope of this article. The design target, however, is the peak load of a national consultation, and every mechanism described above was sized for that load and not for a demonstration.

5 Limitations

A scripted bot gives up open conversation by design: citizens who type free text receive buttons and not comprehension, and the third consecutive unrecognized input ends the exchange. The deterministic script supports auditability but limits how freely participants can express themselves. WhatsApp's own limits also bind the product: the proposal list is capped at what a list message holds, and the remainder is visible only on the site. Identity relies on CPF or GovBR, both Brazilian constructs, so transferring the engine to another jurisdiction means reimplementing the identification adapter. Finally, although the vote path is fully delegated to Decidim core, the script is version-coupled to the host platform's Decidim line, as are all modules in the ecosystem.

6 Related work

Chatbots in e-participation are usually reported as natural-language interfaces, such as retrieval assistants over consultation corpora or dialog systems meant to make institutions "conversational". This case study differs: the bot is an interface to a formal participatory act, and its contribution is institutional fit and not linguistic sophistication. The closest relatives are transactional government chatbots (permit renewals, appointment booking), which chose deterministic menus over open dialogue for the reasons visible here (auditability, error budgets, and the cost of a wrong answer in a formal process) but rarely carry a vote. Within the Decidim ecosystem [3], the engine complements the platform's web-native components and its ephemeral-participation line of development [5], and it follows the ecosystem's no-fork conventions, so the conversation logic never patches core. To our knowledge, no open-source participation platform ships a scripted, multi-tenant, provider-brokered messaging channel that casts votes through the platform's own permission layer, which is the integration that makes chat votes and site votes a single dataset.

7 Availability

decidim-chatbot lives in the Brasil Participativo host application [1] (directory decidim-chatbot/), licensed AGPL-3.0-or-later, and targets Decidim 0.32.x on Ruby ~> 3.4. Runtime requirements are those of the host platform, plus a background job processor (Sidekiq) for the inbox pipeline and credentials for a WhatsApp provider that implements the broker interface. Extraction into a standalone, community-installable gem, as its sibling participatory-texts module already is, is planned, and a permanent DOI will be minted through Zenodo at that release.

8 Conclusion

The architecture is deliberately conservative: a versioned script, a serialized inbox, a one-time identification link, and a vote delegated to the same machinery the website uses, with no language model and no free-form dialogue. In government this conservatism is the main benefit, because the question "what exactly did the system do?" must have an exact answer, and here the answer is a file an auditor can read. In this design the script is the policy and the repository is the record.

References

[1] Brasil Participativo host application, including decidim-chatbot. https://gitlab.com/lappis-unb/decidimbr/participa

[2] Brasil Participativo: module ecosystem on Decidim 0.32.x. https://gitlab.com/lappis-unb/decidimbr

[3] Decidim: free open-source participatory democracy framework. https://decidim.org

[4] SERPRO WhatsApp API documentation. [URL to be added by the authors]

[5] decidim-ephemeral_participation module (Octree). [URL to be added by the authors]

[6] MetaDecidim: community process and proposals. https://meta.decidim.org